

HPC Project Ideas

General Notes:

- Plagiarism is strictly prohibited. So don't copy anyone else's work to be safe and away from applying cheating policies on you
- AI Generated code or code copied from the internet is strictly prohibited
- Your code has to handle the problem in parallel as this will be the primary thing to be evaluated.
- Test your work on a large variety of inputs with different number of threads/processes
- Number of "Processes" / "Threads" should be dynamic and the input size is also dynamic
- You need to follow the project algorithm and pseudocode guide specified in [HPC_Project_Algorithms_and_Pseudocode_Reference.pdf](#)

Your idea should be implemented in:

1. OpenMP
2. MPI
3. Sequential

Deliverables:

1. Project source code.
2. Report (check report requirements below)

Deadlines:

- Project Idea Registration [18/4/2026](#)
- Source Code and Report due [15/5/2026 11:59 PM on LMS](#)
- Discussion/Presentation starting from [16/5/2026](#)

Project 1: Parallel PageRank Implementation

Parallel PageRank for Large-Scale Graphs using MPI and OpenMP

Objective

The goal of this project is to design, implement, and analyze a parallel version of the PageRank algorithm using both **MPI (distributed memory)** and **OpenMP (shared memory)**.

You will explore key High Performance Computing (HPC) concepts including:

- Parallel decomposition
- Inter-process communication
- Synchronization
- Load balancing
- Performance analysis and scalability

Background

PageRank is an algorithm used to rank nodes in a graph based on the structure of incoming links. It is widely used in search engines and network analysis.

The algorithm iteratively updates the rank of each node using:

- Contributions from incoming neighbors
- A damping factor (typically 0.85)
- Iterative convergence until ranks stabilize

Project Description

You are required to implement **three versions** of the PageRank algorithm:

1. Sequential Version

- Implement a baseline PageRank algorithm
- Use a sparse graph representation (adjacency list or CSR preferred)
- Support configurable:
 - Number of nodes
 - Number of iterations or convergence threshold
 - Damping factor

2. OpenMP Parallel Version

- Use shared-memory parallelism
- Parallelize rank updates using OpenMP
- Ensure:
 - Correct synchronization
 - Proper use of reductions
 - No race conditions

3. MPI Parallel Version

- Distribute the graph across multiple processes
- Each process handles a subset of nodes
- Implement communication of rank updates between processes

You may use:

- MPI_Send / MPI_Recv (point-to-point), OR
- Collective operations like MPI_Allreduce, MPI_Allgather

Key challenges include:

- Partitioning graph vertices across processes
- Communicating rank updates between partitions
- Implementing global convergence detection
- Handling sparse adjacency structures efficiently

Deliverables

1. Source Code (required as zip folder)

- Sequential version
- OpenMP version
- MPI version
- Must be well-structured and commented

2. Experimental Results (in report)

Run experiments on:

- Different graph sizes
- Different number of threads/processes

Include:

- Execution time
- Speedup
- Efficiency

3. Performance Analysis (in report)

Discuss:

- Compare MPI and OpenMP scalability
- Communication overhead (MPI)
- Thread efficiency (OpenMP)
- Load balancing
- Bottlenecks
- Time comparison between the 3 approaches

4. Final Report (Required)

Your report must include:

A. Introduction

- Overview of PageRank
- Objectives of the project

B. Implementation Details

- Data structures used
- Parallelization approach (OpenMP + MPI)
- Partitioning strategy

C. Results

- Experimental Results
- Performance analysis

D. Discussion

- Challenges faced
- Debugging strategies
- Observations on performance

E. Conclusion

- Summary of findings
- Possible improvements

Project 2: Parallel N-Body Simulation (Gravitational System)

Objective

Simulate the motion of **N particles interacting under gravity** using **sequential, OpenMP, and MPI implementations**.

You will learn:

- Parallel computation of $O(N^2)$ interactions
- Data sharing and synchronization in MPI
- Load balancing and reduction operations in OpenMP
- Performance analysis and scalability evaluation

Background

The N-Body problem is a classical problem in physics and computational science. It involves simulating the motion of N particles that interact with each other under physical forces, most commonly gravitational forces. Each particle exerts a force on every other particle, and their positions and velocities are updated over time according to Newton's laws of motion.

Project Description

You are required to implement **three versions** of the N-Body simulation:

1. Sequential Version

- Update particle positions and velocities using Newton's law of gravitation
- Compute forces **pairwise** for all particles
- Support configurable:
 - Number of particles
 - Time step
 - Number of iterations

2. OpenMP Parallel Version

- Parallelize force computation across particles
- Use reductions where needed to avoid race conditions
- Ensure thread safety when updating velocities and positions

3. MPI Parallel Version

- Distribute particles among processes
- Exchange particle positions each timestep using MPI collectives (`MPI_Allgather` recommended)
- Ensure synchronization between timesteps

Key HPC Challenges

- High computational complexity: $O(N^2)$ per timestep
- Communication vs computation trade-offs: MPI needs full particle exchange each timestep
- Synchronization: ensure all processes are up-to-date before updating positions
- Thread safety: OpenMP reductions for force sums

Deliverables

1. Source Code (as a zip folder)

- Sequential version
- OpenMP version
- MPI version
- Must be well-structured and commented

2. Experimental Results (in report)

Run experiments on:

- Run for increasing particle counts
- Measure execution time, speedup, and efficiency

3. Performance Analysis (in report)

Discuss:

- Compare MPI and OpenMP scalability
- Communication overhead (MPI)
- Thread efficiency (OpenMP)
- Load balancing
- Bottlenecks
- Time comparison between the 3 approaches

4. Final Report (Required)

Your report must include:

A. Introduction

- Overview of N-Body Simulation
- Objectives of the project

B. Implementation Details

- Data structures used
- Parallelization approach (OpenMP + MPI)

C. Results

- Experimental Results
- Performance analysis

D. Discussion

- Challenges faced
- Debugging strategies
- Observations on performance

E. Conclusion

- Summary of findings
- Possible improvements